



Graphic Era
Hill University
DEHRADUN • BHIMTAL • HALDWANI

PROJECT AND TEAM INFORMATION

Project Title

Community Safety and Incident Tracker

Project GitHub URL

<https://github.com/Arman-Singh7/Community-Safety-Incident-Tracker>

Student/Team Information

Team Name:	Mavericks
Team member 1 (Team Lead) (Arman Singh, University Roll Number: 2318488, student ID: 23011318, email: armansinghpanwar35@gmail.com)	

Team member 2

(Akhil Nautiyal, University Roll Number: 2318258, student ID: 230112637, email: akhil015jan@gmail.com)

**Team member 3**

(Anuj Dobhal, University Roll Number: 2318450 , student ID: 230114797, email: anujdobhal619@gmail.com)

**Team member 4**

(Priyanshu Rawat, University Roll Number: 2319327, student ID: 23011735, email: Priyanshurwt18@gmail.com)



PROJECT PROGRESS DESCRIPTION

Project Abstract

(Brief restatement of your project's main goal. Max 300 words).

The Community Safety and Incident Tracker is a web-based application developed to help communities report, monitor, and manage safety-related incidents efficiently. The system provides secure authentication, role-based access control, incident reporting, attachment management, real-time alerts, messaging, audit logging, and data export/import capabilities.

Users can create, update, and review incidents according to their assigned roles. Administrators have additional privileges to manage data exports, imports, and audit logs. The application uses Server-Sent Events (SSE) to provide real-time notifications and alerts without requiring page refreshes.

The backend is built using Node.js, Express, SQLite, JWT authentication, and bcrypt-based password security. The frontend is implemented using HTML, CSS, and JavaScript, providing a responsive interface with light and dark themes. The system emphasizes security, accountability, and usability while maintaining a clean and user-friendly dashboard.

The project demonstrates practical implementation of web development, database management, authentication, authorization, real-time communication, and software engineering principles. It is suitable for deployment in residential communities, educational institutions, and local organizations that require structured incident tracking and communication facilities.

Updated Project Approach and Architecture

(Describe your current approach, including system design, communication protocols, libraries used, etc. Max 300 words).

The application follows a client-server architecture. The frontend communicates with backend REST APIs using HTTP requests and receives live alerts through Server-Sent Events (SSE). SQLite serves as the primary database for storing users, incidents, messages, alerts, and audit logs.

Authentication is implemented using JWT tokens and bcrypt password hashing. Middleware enforces role-based authorization for viewers, analysts, and administrators. Incident attachments are uploaded through Multer and stored securely with metadata maintained in the database.

The backend is divided into modular components including authentication, incident management, alerts, messaging, auditing, and export/import services. This modular design improves maintainability and scalability.

Technologies used include Node.js, Express, better-sqlite3, bcryptjs, jsonwebtoken, multer, dotenv, HTML, CSS, JavaScript, SQLite, REST APIs, and SSE. The frontend provides responsive layouts, dashboard navigation, role-aware interfaces, and theme support. The architecture ensures secure communication, efficient data handling, and real-time updates.

Tasks Completed

(Describe the main tasks that have been assigned and completed. Max 250 words).

Task Completed	Team Member
Database schema design and setup	Akhil Nautiyal
Authentication and authorization module	Akhil Nautiyal
JWT security middleware implementation	Akhil Nautiyal
Incident CRUD APIs	Anuj Dobhal
Attachment upload management	Anuj Dobhal
Incident filtering and search	Anuj Dobhal
Real-time alert streaming using SSE	Priyanshu Rawat
Messaging system development	Priyanshu Rawat
Audit logging and export/import features	Priyanshu Rawat
Frontend dashboard implementation	Arman Singh
API integration and UI testing	Arman Singh
Theme toggle and responsive design	Arman Singh

Challenges/Roadblocks

(Describe the challenges that you have faced. Max 300 words).

Several challenges were encountered during development. Implementing secure authentication required careful integration of JWT-based session management and password hashing. Managing role-based permissions across multiple modules was complex because different users have different access levels.

File upload handling posed challenges related to validation, storage, and security. These issues were addressed by limiting file sizes and storing attachment metadata separately.

Implementing Server-Sent Events for real-time alerts required managing persistent client connections and ensuring reliable message delivery. Exporting data in CSV format required proper escaping and formatting to prevent corruption.

Frontend integration was another challenge because multiple backend modules had to work seamlessly through a single dashboard interface. Responsive design and role-specific UI rendering required extensive testing across different scenarios.

Tasks Pending

(Describe any task that you still need to complete. Max 250 words).

Tasks Pending
Tasks Pending Final integration testing User acceptance testing Documentation refinement GitHub repository cleanup Final bug fixes and optimization Deployment preparation PPT and demonstration video finalization Viva preparation and rehearsal

Project Outcome/Deliverables

(Describe what are the key outcomes / deliverables of the project. Max 200 words).

The project successfully delivers a secure and feature-rich Community Safety and Incident Tracker application. Major deliverables include: • Secure authentication and authorization system • Incident reporting and management module • File attachment support • Real-time alert broadcasting using SSE • Internal messaging system • Audit logging functionality • Data export and import features • Responsive frontend dashboard • API documentation and testing reports • Project documentation and presentation materials

Progress Overview

(Summarize how much of the project is done, and what's left. Max 200 words.)

Approximately 90–95% of the project has been completed. Core functionalities such as authentication, incident management, messaging, audit logging, export/import, and real-time alerts are fully implemented. Frontend integration is largely complete and operational. Remaining work primarily includes final testing, documentation polishing, deployment preparation, and presentation refinement. The project is currently in the stabilization and validation phase and is on track for successful submission and demonstration.

Testing and Validation Status

(Provide information about any tests conducted)

Test Type	Status	Notes
Authentication Testing	Pass	Login, logout, token validation verified
Authorization Testing	Pass	Role restrictions working correctly
Incident CRUD Testing	Pass	Create, read, update, delete validated
File Upload Testing	Pass	Attachment handling successful
Messaging Testing	Pass	Inbox and send functionality working
SSE Alert Testing	Pass	Real-time alerts received correctly
Export/Import Testing	Pass	JSON and CSV operations verified
Frontend Integration Testing	Pass	APIs integrated successfully
Responsive UI Testing	Pass	Works on desktop and mobile screens
Security Testing	Pass	Route protection and JWT validation verified

Deliverables Progress

(Summarize the current status of all key project deliverables mentioned earlier. Indicate whether each deliverable is completed, in progress, or pending.)

Deliverables Progress

The project deliverables have reached an advanced stage of completion. Core backend components including **database.js**, **auth.js**, authentication middleware, and the security framework have been fully developed and tested. The **Incident Management Module**, including CRUD operations, attachment handling, filtering, and role-based access controls, has been successfully implemented. The real-time communication components such as **alerts.js**, **messages.js**, **audit.js**, and **export.js** are complete and functioning as expected. On the frontend side, **index.html**, **style.css**, and **script.js** have been integrated with backend APIs, providing a responsive dashboard with role-based access, theme support, and live alert updates. API documentation, module-level testing, and most technical documentation have also been completed. Currently, the team is focused on final integration testing, documentation refinement, README updates, presentation slides, and deployment preparation. Overall, approximately 90–95% of the planned deliverables have been completed, with the remaining tasks primarily related to validation, optimization, and final project presentation.